SECURE IoT-BASED MEDICAL DEVICE MONITORING SYSTEM

PROJECT **REPORT**	**PROJECT REPORT SUBMITTED** IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF BACHELOR OF ENGINEERING IN **COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)** OF THE ANNA UNIVERSITY
PROJECT **WORK**	**SUBMITTED BY** **LEKSHME BALAJE R – 722825149035**
BATCH	**2025 – 2029**
	UNDER THE GUIDANCE OF **Dr. SURIYA M, M.E.,** **ASSISTANT PROFESSOR** Department of Computer Science and Engineering
COMPUTER SCIENCE AND ENGINEERING **Sri Eshwar College of Engineering** (An Autonomous Institution – Affiliated to Anna University) **COIMBATORE – 641 202**	

BONAFIDE CERTIFICATE

Certified that this report titled "SECURE IoT-BASED MEDICAL DEVICE MONITORING SYSTEM USING DJANGO AND MONGODB" is the bonafide work of LEKSHME BALAJE R – 722825149035 who carried out the project work under my supervision

SIGNATURE **SIGNATURE**

Dr. R. SUBHA M.E., Ph.D., **HEAD OF THE DEPARTMENT** PROFESSOR, COMPUTER SCIENCE AND ENGINEERING, Sri Eshwar College of Engineering, Coimbatore-641202.	**Dr. SURIYA M, M.E.,** **SUPERVISOR** ASSISTANT PROFESSOR, COMPUTER SCIENCE AND ENGINEERING, Sri Eshwar College of Engineering, Coimbatore-641202.

Submitted for the Autonomous Semester End Project Viva-Voce held on

. . . . . .

(Internal Examiner) **(External Examiner)**

Sri Eshwar College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution

QUALITY POLICY

To establish a system of Quality Enhancement, which would on a continuous basis evaluate and enhance the quality of teaching – learning, research and extension activities of the institution, leading to improvements in all processes, enabling the institution to attain excellence.

INSTITUTE VISION

To be recognized as a premier institution, grooming students into globally acknowledged engineering professionals.

INSTITUTE MISSION

- Providing outcome and value-based engineering education Nurturing research and entrepreneurial culture.
- Enabling students to be industry ready and fulfil their career aspirations
- Grooming students through behavioural and leadership training programs
- Making students socially responsible

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT VISION

To groom students into globally competent software professionals and meet the ever-changing requirements of the industry.

DEPARTMENT MISSION

- Creating a quality academic environment with relevant IT infrastructure and empowering faculty and students with emerging technologies.
- Motivating staff and students to actively involve in lifelong learning and fostering research.
- Inculcating leadership and entrepreneurship skills in students.

- Generating opportunities for students to evolve as competent software professionals with societal consciousness.

PROGRAM EDUCATIONAL OBJECTIVES

PEO1: To prepare graduates for a career in software engineering and cybersecurity domains.

PEO2: To prepare students for higher studies, research, entrepreneurial and leadership roles by imparting the quality of lifelong learning.

PEO3: To enable students to apply innovative solutions for real-life problems in computer science and cyber security domain.

PROGRAM OUTCOMES

- PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop the solution of complex engineering problems.
- PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.
- PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for public health and safety, culture, society, and environment.
- PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.
- PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.
- PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability.
- PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.
- PO8: Individual and Collaborative Teamwork: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
- PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large.
- PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making.

- PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for independent and life-long learning, adaptability to new and emerging technologies, and critical thinking.

PROGRAM SPECIFIC OUTCOMES

- PSO1: Demonstrate knowledge in open-source technologies and cybersecurity frameworks.
- PSO2: Develop innovative solutions by adapting emerging technologies for industry-oriented and security-focused applications.
- PSO3: Implement SDLC principles and security-by-design for project/product development.

Sri Eshwar
College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution

ACKNOWLEDGEMENT

I thank God for his grace and blessing that have been showered upon me to bring success for this project. The success I have accomplished in my project would not have been possible without the timely help and guidance rendered to me by many people. I take this opportunity to thank them all.

I wish to express my deep gratitude to Mr. R. Mohanram, Chairman, Sri Eshwar College of Engineering, Kondampatti for being kindly permitted me to carry on the project. He has an abiding interest in our wellbeing.

I am grateful to Dr. Sudha Mohanram M.E., Ph.D., Principal, Sri Eshwar College of Engineering, Kondampatti for having made all provisions available to me.

I am thankful to Mr. R. Rajaram, Director, Sri Eshwar College of Engineering, Kondampatti for all his support to complete the project effectively.

I would like to express my heartfelt gratitude and humble thanks to Head of the Department Dr. R. SUBHA M.E., Ph.D., for having facilitated me to complete the project successfully.

With the deep sense of gratitude, I extend my earnest and sincere thanks to my project guide Dr. SURIYA M, M.E.,, Assistant Professor, Department of Computer Science and Engineering, for her kind suggestion and co-operation for doing the project. Finally, I would like to extend my thanks to my teaching and non-teaching staffs those who helped me in bringing out project successfully.

Sri Eshwar College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution

TABLE OF CONTENTS

CHAPTER NO.		TITLE	PAGE NO.
		ABSTRACT	i
		LIST OF FIGURES	ii
1		INTRODUCTION	
	1.1	OBJECTIVES	1
	1.2	SCOPE OF THE PROJECT	1
2		SYSTEM ANALYSIS AND SPECIFICATION	
	2.1	PROBLEM DESCRIPTION	2
	2.2	FUNCTIONAL REQUIREMENTS	3
	2.3	SOFTWARE AND HARDWARE REQUIREMENTS	4
	2.4	NON-FUNCTIONAL REQUIREMENTS	4
3		PROJECT DESCRIPTION	
	3.1	MODULE DESCRIPTION	5
4		IMPLEMENTATION	7
5		RESULTS AND DISCUSSION	13
6		CONCLUSION & FUTURE ENHANCEMENT	16
		REFERENCES	17

Sri Eshwar College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution

ABSTRACT

This project presents the design and implementation of a Secure IoT-Based Medical Device Monitoring System aimed at enhancing the safety and reliability of patient data in hospital environments. Modern healthcare systems increasingly rely on interconnected medical devices such as heart rate monitors, temperature sensors, and SpO_2 sensors to continuously collect and transmit patient data. However, these systems are highly vulnerable to cyber threats such as data interception, device spoofing, and unauthorized access.

The proposed system uses Python with the Django framework for backend development, MongoDB as the NoSQL database for storing patient and device data, and HTML/CSS with Chart.js for building a real-time, responsive web interface. IoT devices are simulated using Python scripts that generate real-time patient vital data and transmit it securely to the server.

To ensure security, the system incorporates multiple cybersecurity mechanisms. All data transmitted between devices and the server is encrypted using AES-256-CBC to prevent unauthorized interception. Each device is authenticated using unique HMAC-SHA256 tokens to ensure that only trusted devices can communicate with the system. Data integrity is maintained using SHA-256 hashing to detect any tampering during transmission. The system also includes an anomaly detection module that identifies suspicious activities such as abnormal vital-sign patterns or unauthorized device access, triggering real-time alerts for administrators.

The developed dashboard allows healthcare staff to monitor patient vitals in real time, view device status, manage security alerts, and review a complete audit trail of all system events. This project demonstrates how integrating IoT with robust cybersecurity practices can significantly improve the safety, integrity, and reliability of digital healthcare systems, providing a scalable and secure solution for modern hospital environments.

Sri Eshwar College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution

LIST OF FIGURES

FIG.NO	TITLE	PAGE NO.
5.1	Login Page – MedIoT Monitor Dashboard	13
5.2	Real-Time Monitoring Dashboard with Live Charts	13
5.3	Device Registry and Registration Page	14
5.4	Security Alerts Management Panel	14
5.5	Audit Log – Security Event Trail	15
5.6	IoT Simulator Terminal Output	15

CHAPTER 1

INTRODUCTION

1.1 OBJECTIVES

The Secure IoT-Based Medical Device Monitoring System aims to develop an intelligent, real-time monitoring platform that collects patient vital data from IoT-enabled hospital devices and transmits it securely to a centralized server. The system integrates multiple cybersecurity mechanisms — including AES-256-CBC encryption, HMAC-SHA256 device authentication, and SHA-256 data integrity verification — to protect sensitive medical information from threats such as interception, device spoofing, and data tampering.

By leveraging Python, Django, and MongoDB, the system provides a scalable backend capable of handling continuous real-time data streams from multiple devices simultaneously. An automated anomaly detection engine monitors incoming vitals and immediately triggers alerts when values fall outside clinically normal ranges, enabling rapid clinical response.

The interactive web dashboard empowers healthcare administrators and clinical staff to monitor patient data in real time, oversee device connectivity, manage security incidents, and audit all system events. By combining IoT technology with enterprise-grade cybersecurity practices, this project delivers a practical, hospital-ready solution that significantly improves the integrity and reliability of digital healthcare infrastructure.

1.2 SCOPE OF THE PROJECT

The scope of this project encompasses the following key areas:

- Design and implementation of a secure IoT data pipeline from simulated medical devices to a Django-based central server.
- Integration of AES-256-CBC encryption, HMAC-SHA256 device authentication, and SHA-256 data integrity checking for all device communications.

- Development of a real-time web dashboard with live Chart.js visualizations polling every five seconds.
- Automated anomaly detection across seven vital-sign parameters with instant alert generation and resolution management.
- A comprehensive audit logging system tracking all system events, login activity, device registrations, and security incidents.
- A MongoDB-backed NoSQL storage system with an automatic in-memory fallback for environments without a live MongoDB instance.
- A Python IoT device simulator capable of generating realistic, randomized patient vital data including intentional anomalies and rogue device attacks for testing.

The project deliberately excludes real physical hardware integration, live patient monitoring certification, HL7/FHIR interoperability standards, mobile application development, and real-time chat functionality, in order to maintain focus on the core security and monitoring objectives within the Python Programming course scope.

CHAPTER 2

SYSTEM ANALYSIS AND SPECIFICATION

2.1 PROBLEM DESCRIPTION

In contemporary hospital environments, medical devices such as cardiac monitors, pulse oximeters, blood pressure monitors, and temperature sensors operate as largely isolated systems. Patient data collected by these devices is often recorded manually or stored locally on proprietary hardware, making it difficult to aggregate, monitor, or share in real time. In larger facilities where network-connected devices do exist, communications frequently occur over unencrypted channels, exposing sensitive patient data to potential interception.

Existing IoT-based healthcare platforms face several critical shortcomings. Device authentication is weak or entirely absent, allowing unauthorized or rogue devices to inject false readings into clinical systems. Data transmitted between devices and central servers is rarely encrypted end-to-end, leaving it vulnerable to man-in-the-middle attacks. Audit trails are incomplete or non-existent, making it difficult to investigate security incidents after they occur. Furthermore, anomaly detection — the automated identification of abnormal patient vitals or suspicious device behaviour — is typically not integrated, requiring manual clinical observation that can delay response to life-threatening conditions.

This project addresses all these gaps by implementing a layered security architecture combining encryption, authentication, hashing, anomaly detection, and audit logging within a unified Django-based monitoring platform.

2.2 FUNCTIONAL REQUIREMENTS

2.2.1 DEVICE AUTHENTICATION AND REGISTRATION

- Register IoT devices with unique HMAC-SHA256 tokens.
- Reject all unauthenticated or rogue device connection attempts.
- Maintain device registry with status (online/offline/registered) and last-seen timestamps.

2.2.2 SECURE DATA TRANSMISSION

- Encrypt all device-to-server payloads using AES-256-CBC.
- Verify SHA-256 integrity hash for every received reading.
- Reject and alert on any payload with a mismatched hash.

2.2.3 ANOMALY DETECTION AND ALERTING

- Monitor seven vital-sign parameters: heart rate, temperature, SpO_2, systolic BP, diastolic BP, respiratory rate, and glucose.
- Automatically generate alerts for any reading outside clinically normal ranges.
- Allow administrators to resolve alerts and track resolution history.

2.2.4 REAL-TIME DASHBOARD

- Display live Chart.js visualizations of patient vitals, refreshing every five seconds.
- Show device connectivity status, active alert count, and total readings in summary cards.

2.2.5 AUDIT LOGGING

- Record all system events: logins, logouts, device registrations, data transmissions, alert creations, and alert resolutions.
- Display audit log with timestamp, event type, description, user, and severity level.

2.3 SOFTWARE AND HARDWARE REQUIREMENTS

2.3.1 HARDWARE REQUIREMENTS

- Minimum: Intel i5 processor, 8 GB RAM, 50 GB storage
- Recommended: Intel i7 or higher, 16 GB RAM, 100 GB SSD for optimal performance

2.3.2 SOFTWARE REQUIREMENTS

- Python 3.10+
- Django Framework 4.2
- MongoDB 6.0 (optional – in-memory fallback included)
- PyMongo 4.0+
- PyCryptodome 3.18+ (for AES-256-CBC)
- Requests library (for IoT simulator)
- Chart.js 4 (CDN)

2.4 NON-FUNCTIONAL REQUIREMENTS

2.4.1 SECURITY

- All device communications must be encrypted with AES-256-CBC.
- Device identity must be verified using HMAC-SHA256 on every request.
- Data integrity must be verified using SHA-256 before storage.
- Protect against common web vulnerabilities (XSS, CSRF) using Django built-in protections.

2.4.2 PERFORMANCE

- Dashboard must load and refresh within 2-3 seconds.
- Server must handle concurrent device transmissions efficiently.
- MongoDB queries must be indexed for fast retrieval of recent readings.

2.4.3 SCALABILITY

- MongoDB document model must accommodate additional vital parameters without schema migration.
- Device simulator must be extensible to add more virtual devices.

2.4.4 RELIABILITY

- System must operate fully using the in-memory fallback when MongoDB is unavailable.
- Failed device authentications must be logged and alerted without crashing the server.

2.4.5 USABILITY

- Dashboard must be navigable with a fixed sidebar and consistent page layout.
- All alert and audit tables must be sortable and clearly labelled.

CHAPTER 3

PROJECT DESCRIPTION

3.1 MODULE DESCRIPTION

3.1.1 SECURITY MODULE (security.py)

The Security Module is the backbone of the entire system, implementing four distinct cybersecurity mechanisms. The AES-256-CBC encryption/decryption functions use PyCryptodome with a randomly generated 16-byte IV per message to ensure ciphertext uniqueness. The HMAC-SHA256 device token generator and verifier ensure that only registered devices with valid tokens can submit data. The SHA-256 hashing functions create and verify data integrity digests to detect any in-transit tampering. Finally, the anomaly detection engine checks seven patient vital-sign parameters against pre-defined clinical normal ranges and returns detailed descriptions of any violations. All security events are written to the audit log through the log_audit_event() utility.

3.1.2 DEVICE MANAGEMENT MODULE (views.py – register_device)

The Device Management Module handles the registration of new IoT devices. When a device is registered through the web interface or API, the server generates a unique HMAC-SHA256 token for that device and stores the device profile in MongoDB, including its device ID, type, location, and token. The module also updates the device status to "online" whenever a valid reading is received and records the last-seen timestamp. Device details are displayed in the Device Registry dashboard page, giving administrators a live overview of all connected and registered medical equipment.

3.1.3 DATA INGEST MODULE (views.py – receive_data)

The Data Ingest Module implements the five-step security pipeline that processes every incoming IoT reading. Step 1 verifies the HMAC-SHA256 device token and rejects unauthorized devices with a 401 response. Step 2 decrypts the AES-256-CBC payload. Step 3 verifies the SHA-256 integrity hash against the decrypted data. Step 4 runs the anomaly detection engine and generates alerts for abnormal readings. Step 5 persists the validated reading to the MongoDB "readings" collection and updates the device status. Failures at any step are logged to the audit trail and may generate security alerts.

3.1.4 ALERT MANAGEMENT MODULE (views.py – alerts_view, resolve_alert)

The Alert Management Module provides a real-time view of all active and historical security and anomaly alerts. Alerts are categorized by type (ANOMALY_DETECTED, UNAUTHORIZED_ACCESS, DATA_TAMPERING) and severity (warning, critical). Administrators can resolve open alerts through the dashboard, which marks them as resolved with a timestamp and records the resolution action in the audit log. The dashboard's summary card always shows the current count of unresolved alerts, enabling rapid incident awareness.

3.1.5 REAL-TIME DASHBOARD MODULE (dashboard.html + api_readings)

The Real-Time Dashboard Module combines a Django backend API with a Chart.js frontend to deliver live patient data visualizations. The /api/readings/ endpoint returns the most recent 30 readings in JSON format. The dashboard JavaScript polls this endpoint every five seconds and updates two line charts: a Heart Rate chart and a combined SpO_2 and Temperature chart. The dashboard also polls /api/stats/ to refresh the four summary statistics cards (total devices, online devices, active alerts, total readings). The module uses a dark industrial design theme with a fixed sidebar navigation for consistent access to all system views.

3.1.6 IoT DEVICE SIMULATOR MODULE (simulator.py)

The IoT Device Simulator Module models five virtual hospital devices: a Multi-Parameter Monitor (ICU Bed 1), a Heart Rate Monitor (Ward A Bed 3), an SpO_2 Sensor (Ward B Bed 7), a Temperature Sensor (Recovery Bed 2), and a Blood Pressure Monitor (ICU Bed 2). On startup, each device registers itself with the server and receives its HMAC token. In the main simulation loop, each device generates patient vitals — with approximately 15% probability of anomalous values — applies the full client-side security stack (AES encrypt → SHA-256 hash → HMAC token), and transmits the payload via HTTP POST. Every fifth round, the simulator attempts a rogue device connection with an invalid token to verify the server correctly rejects unauthorized access.

3.1.7 AUDIT LOG MODULE (audit_log_view + log_audit_event)

The Audit Log Module maintains a complete, timestamped record of every significant system event. Events recorded include user logins and logouts, device registrations, data receptions (successful and failed), integrity violations, unauthorized access attempts, anomaly detections, and alert resolutions. Each log entry captures the event type, description, user or system actor, severity level, and UTC timestamp. The audit log view displays the 100 most recent events in reverse chronological order and is accessible only to authenticated administrators.

CHAPTER 4

IMPLEMENTATION

settings.py

```
MONGODB_URI = os.environ.get("MONGODB_URI", "mongodb://localhost:27017/")
MONGODB_DB_NAME = "iot_medical_db"

# AES-256 Encryption Key (32 bytes)
ENCRYPTION_KEY = os.environ.get("ENCRYPTION_KEY",
                 "medicalIoTSecure32ByteKey1234567")

# Device HMAC Secret
DEVICE_SECRET = os.environ.get("DEVICE_SECRET",
                "iot-device-shared-secret-2024")

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.sessions",
    "monitor",
]

LOGIN_URL = "/login/"
LOGIN_REDIRECT_URL = "/"
```

db.py (MongoDB with in-memory fallback)

```
try:
    from pymongo import MongoClient
    _client = MongoClient(settings.MONGODB_URI,
                          serverSelectionTimeoutMS=2000)
    _client.server_info()   # Raises if MongoDB unreachable
    _db = _client[settings.MONGODB_DB_NAME]
    MONGO_AVAILABLE = True
except Exception:
    MONGO_AVAILABLE = False  # Falls back to in-memory store

def get_collection(name):
    if MONGO_AVAILABLE:
        return _db[name]
    return _mem_col(name)   # PyMongo-compatible fallback

def get_devices_col():   return get_collection("devices")
def get_readings_col():  return get_collection("readings")
def get_alerts_col():    return get_collection("alerts")
def get_audit_logs_col():return get_collection("audit_logs")
```

security.py (AES-256 Encryption)

```python
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

def encrypt_data(data: dict) -> str:
    plaintext = json.dumps(data).encode("utf-8")
    key = _get_key()           # 32-byte AES-256 key
    iv  = os.urandom(16)       # New random IV per message
    cipher = AES.new(key, AES.MODE_CBC, iv)
    ct = cipher.encrypt(pad(plaintext, AES.block_size))
    return base64.b64encode(iv + ct).decode("utf-8")

def decrypt_data(encrypted: str) -> dict:
    raw = base64.b64decode(encrypted)
    iv, ct = raw[:16], raw[16:]
    cipher = AES.new(_get_key(), AES.MODE_CBC, iv)
    return json.loads(unpad(cipher.decrypt(ct), 16))
```

security.py (HMAC Device Authentication)

```python
def generate_device_token(device_id: str) -> str:
    secret = settings.DEVICE_SECRET.encode("utf-8")
    return hmac.new(secret, device_id.encode(),
                    hashlib.sha256).hexdigest()

def verify_device_token(device_id: str, token: str) -> bool:
    expected = generate_device_token(device_id)
    # Constant-time comparison prevents timing attacks
    return hmac.compare_digest(expected, token)

def hash_data(data: dict) -> str:
    s = json.dumps(data, sort_keys=True).encode()
    return hashlib.sha256(s).hexdigest()

def verify_integrity(data: dict, expected_hash: str) -> bool:
    return hmac.compare_digest(hash_data(data), expected_hash)
```

security.py (Anomaly Detection)

```python
NORMAL_RANGES = {
    "heart_rate":                (40,  180),  # bpm
    "temperature":              (35.0, 40.0), # Celsius
    "spo2":                     (90,  100),   # %
    "blood_pressure_systolic":  (80,  180),   # mmHg
    "blood_pressure_diastolic": (40,  120),   # mmHg
    "respiratory_rate":         (8,   30),    # breaths/min
    "glucose":                  (50,  400),   # mg/dL
}

def detect_anomaly(data: dict) -> tuple[bool, list[str]]:
    reasons = []
    for metric, (lo, hi) in NORMAL_RANGES.items():
        value = float(data.get(metric, lo))
        if not (lo <= value <= hi):
            reasons.append(
              f"{metric} = {value} (normal {lo}-{hi})")
    return bool(reasons), reasons
```

views.py (Five-Step Security Pipeline)

```python
@csrf_exempt
@require_http_methods(["POST"])
def receive_data(request):
    payload   = json.loads(request.body)
```

```
    device_id = payload["device_id"]

    # Step 1 - Device Authentication
    if not verify_device_token(device_id, payload["token"]):
        log_audit_event("AUTH_FAILURE", ..., severity="critical")
        _create_alert("UNAUTHORIZED_ACCESS", ..., "critical")
        return JsonResponse({"error": "Authentication failed"},
                            status=401)

    # Step 2 - Decrypt AES-256-CBC payload
    data = decrypt_data(payload["encrypted"])

    # Step 3 - Verify SHA-256 integrity hash
    if not verify_integrity(data, payload["hash"]):
        _create_alert("DATA_TAMPERING", ..., "critical")
        return JsonResponse({"error": "Integrity check failed"},
                            status=400)

    # Step 4 - Anomaly Detection
    is_anomaly, reasons = detect_anomaly(data)
    if is_anomaly:
        _create_alert("ANOMALY_DETECTED", ..., "warning")

    # Step 5 - Persist to MongoDB
    get_readings_col().insert_one({
        "device_id": device_id, "data": data,
        "hash": payload["hash"], "is_anomaly": is_anomaly,
        "timestamp": datetime.now(timezone.utc).isoformat(),
    })
    get_devices_col().update_one(
        {"device_id": device_id},
        {"$set": {"status": "online",
                  "last_seen": datetime.now(timezone.utc).isoformat()}},
    )
    return JsonResponse({"status": "ok", "anomaly": is_anomaly})
```

simulator.py (IoT Device Simulator)

```
DEVICES = [
  {"device_id":"DEV-001","device_type":"Multi-Parameter Monitor",
   "location":"ICU - Bed 1"},
  {"device_id":"DEV-002","device_type":"Heart Rate Monitor",
   "location":"Ward A - Bed 3"},
  {"device_id":"DEV-003","device_type":"SpO2 Sensor",
   "location":"Ward B - Bed 7"},
  {"device_id":"DEV-004","device_type":"Temperature Sensor",
   "location":"Recovery - Bed 2"},
  {"device_id":"DEV-005","device_type":"BP Monitor",
   "location":"ICU - Bed 2"},
]

def send_reading(self, anomaly=False):
    vitals    = generate_vitals(anomaly=anomaly)
    encrypted = encrypt_data(vitals)    # AES-256-CBC
    hash_val  = hash_data(vitals)       # SHA-256
    payload = {
        "device_id": self.device_id,
        "token":     self.token,        # HMAC-SHA256
        "encrypted": encrypted,
        "hash":      hash_val,
    }
    resp = requests.post(SERVER+"/api/receive-data/",
                         json=payload)
    return resp.status_code == 200

# ~15% of readings are anomalous; every 5th round
# simulates a rogue device with an invalid token.
```

CHAPTER 5

RESULTS AND DISCUSSION

LOGIN PAGE:

The login page shown in Fig 5.1 provides a secure, dark-themed authentication interface for hospital administrators. It features username and password input fields with Django CSRF protection. The page displays security badges indicating AES-256, HMAC authentication, SHA-256 hashing, and anomaly detection are active. Only authenticated Django users may access any page of the monitoring dashboard.

[Fig 5.1 – Login Page – MedIoT Monitor Dashboard]

REAL-TIME MONITORING DASHBOARD:

The main dashboard shown in Fig 5.2 displays four live summary statistics cards: Total Devices, Online Devices, Active Alerts, and Total Readings. Below the cards, two Chart.js line charts show the most recent 30 readings: the left chart displays heart rate (bpm) and the right chart shows SpO_2 (%) and temperature (°C) on a dual-axis. The charts automatically refresh every five seconds. Below the charts, a device status table shows all registered devices with their type, location, and current online/offline status, alongside a Recent Alerts panel listing the five most recent unresolved alerts.

[Fig 5.2 – Real-Time Monitoring Dashboard with Live Charts]

DEVICE REGISTRY PAGE:

The Device Registry page shown in Fig 5.3 allows administrators to register new IoT devices by specifying a device ID, type, and hospital location. Upon registration, the server generates and displays the HMAC-SHA256 authentication token for that device. The page also lists all currently registered devices with their partial token (first 16 characters), registration date, last-seen timestamp, and status badge. This provides full visibility of the hospital's IoT device inventory.

[Fig 5.3 – Device Registry and Registration Page]

SECURITY ALERTS PANEL:

The Security Alerts page shown in Fig 5.4 lists all active and historical alerts with timestamp, type (ANOMALY_DETECTED, UNAUTHORIZED_ACCESS, DATA_TAMPERING), full message, associated device ID, and severity badge (warning/critical). Unresolved alerts display a Resolve button. When an administrator resolves an alert, the row is immediately updated in the UI and the resolution event is written to the audit log. Testing with the simulator confirmed that rogue device connections generate UNAUTHORIZED_ACCESS critical alerts, and anomalous vitals generate ANOMALY_DETECTED warning alerts within one second of transmission.

[Fig 5.4 – Security Alerts Management Panel]

AUDIT LOG VIEW:

The Audit Log page shown in Fig 5.5 displays the 100 most recent system events in reverse chronological order. Columns include timestamp, event type (LOGIN, DEVICE_REGISTERED, AUTH_FAILURE, ANOMALY_DETECTED, ALERT_RESOLVED, etc.), description, user or system actor, and severity badge. This view provides a complete, tamper-evident trail of all actions taken within the system, supporting post-incident investigation and regulatory compliance requirements.

[Fig 5.5 – Audit Log – Security Event Trail]

IoT SIMULATOR TERMINAL OUTPUT:

Fig 5.6 shows the simulator terminal during a typical run. Five devices register and begin transmitting encrypted readings every five seconds. Normal readings are logged with heart rate, temperature, and SpO_2 values alongside an "OK" status. Anomalous readings are flagged with a "⚠ ANOMALY" tag and the specific out-of-range values. Every fifth round, the rogue device DEV-ROGUE-999 attempts connection with an invalid token and the server correctly responds with HTTP 401, confirming that the authentication layer is functioning as designed.

[Fig 5.6 – IoT Simulator Terminal Output]

CHAPTER 6

CONCLUSION & FUTURE ENHANCEMENT

The Secure IoT-Based Medical Device Monitoring System, developed using Python, Django, MongoDB, and HTML/CSS/JavaScript, successfully demonstrates how IoT technology and cybersecurity can be effectively integrated to build a reliable, real-time patient monitoring platform for modern hospital environments.

The system's layered security architecture — combining AES-256-CBC encryption, HMAC-SHA256 device authentication, SHA-256 data integrity verification, automated anomaly detection, and comprehensive audit logging — addresses all major threat vectors present in typical hospital IoT deployments. The five-step server-side security pipeline ensures that only data from authenticated, trusted devices reaches the database, and any tampering or unauthorized access attempt is detected and recorded immediately.

The real-time web dashboard provides clinical staff with live visibility into patient vitals, device connectivity, and security events, supporting faster decision-making and reducing response times to clinical anomalies. The MongoDB-backed storage system, with its automatic in-memory fallback, demonstrates practical resilience and ease of deployment across diverse hospital IT environments.

The IoT device simulator validates all security mechanisms end-to-end, including anomaly triggering, rogue device rejection, and integrity failure detection, confirming that the system behaves correctly under adversarial conditions.

Looking ahead, the system can be significantly enhanced with:

- Integration with real physical IoT hardware using MQTT or CoAP protocols for true hospital-grade deployment.
- Implementation of TLS/HTTPS for transport-layer encryption in addition to payload-level AES-256.
- Role-based access control differentiating clinical staff, administrators, and auditors.

- Machine learning-based predictive analytics to anticipate patient deterioration before vitals reach critical thresholds.
- Mobile application support for on-the-go monitoring by clinicians.
- HL7/FHIR interoperability standards for integration with existing Electronic Health Record (EHR) systems.
- Digital Rights Management (DRM) and HIPAA/DPDP Act 2023 compliance certification.
- Multi-hospital scalability using cloud deployment on AWS or Azure with auto-scaling MongoDB Atlas.

Overall, this project demonstrates a practical and extensible approach to securing IoT-based healthcare infrastructure, and highlights the critical importance of embedding cybersecurity at every layer of modern medical device ecosystems.

Sri Eshwar College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution

REFERENCES

Django Framework:

- https://docs.djangoproject.com/en/4.2/
- https://docs.djangoproject.com/en/4.2/topics/db/models/
- https://docs.djangoproject.com/en/4.2/topics/http/views/
- https://docs.djangoproject.com/en/4.2/topics/templates/

MongoDB & PyMongo:

- https://www.mongodb.com/docs/
- https://pymongo.readthedocs.io/

PyCryptodome (AES-256):

- https://pycryptodome.readthedocs.io/

NIST AES Standard:

- FIPS PUB 197: Advanced Encryption Standard (AES). NIST, 2001.

HMAC RFC:

- Krawczyk, H., Bellare, M., & Canetti, R. (1997). RFC 2104. IETF.

Chart.js:

- https://www.chartjs.org/docs/latest/

OWASP Top Ten:

- https://owasp.org/www-project-top-ten/

IoT Healthcare Security:

- Al-Fuqaha et al. (2015). Internet of Things: A Survey. IEEE Communications Surveys & Tutorials, 17(4), 2347-2376.